

The 3 R's in Python

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-04-14

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git collaboration	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: The Python ecosystem	21/4	Y	RZ D8
10: cont. L9 & project feedback	28/4	Y	RZ D8
11: Testing and debugging	5/5	N	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment and peer review **today**.

Today's learning objectives

1. Understand the **3 R's** framework: Readable, Reusable, reproducible
2. Use Python's four main **data structures**: list, dict, tuple, set
3. Write and call **functions** with parameters, defaults, and return values
4. Document functions with **docstrings**
5. Read and write **files** in Python

Contents

- The 3 R's 4
- Data Structures 8
- Functions 29
- Variable scope 39
- File Handling 43
- Putting it all together 50
- Recap 55

The 3 R's

Readable, Reusable, reproducible

The 3 R's

The three qualities that separate *working* code from *good* code:

Readable

Code is read far more often than it is written. Make it easy for others, and future you.

Reusable

Don't repeat yourself. Write it once, use it everywhere.

Reproducible

Produce the same results for the same input and allow others to replicate.

You may write code that **works**. But is it **good**? And most importantly, can you **understand** it, **modify** it, and **trust** it?

```
# Code works, but...  
def f(x):  
    r = []  
    for i in x:  
        if i > 0:  
            r.append(i)  
    return r
```

```
# Same logic, with the 3 R's
def filter_positive(values: list[float]) -> list[float]:
    """Return only the positive values from the input list."""
    return [v for v in values if v > 0]
```

When you generate code with AI, the 3 R's are your checklist for reviewing it.

Data Structures

Python has four built-in **collection** types. Each serves a different purpose:

```
my_list   = [1, 2, 3]           # list – ordered, mutable
my_tuple  = (1, 2, 3)           # tuple – ordered, immutable
my_set    = {1, 2, 3}           # set – unordered, unique items
my_dict   = {"x": 1, "y": 2, "z": 3} # dict – key-value pairs
```

	List	Tuple	Set	Dict
Syntax <i>how to create it</i>	[...]	(...)	{...}	{k: v}
Ordered <i>maintains insertion order</i>	Yes	Yes	No	Yes
Mutable <i>can be modified in place</i>	Yes	No	Yes	Yes
Duplicates <i>allows repeated values</i>	Yes	Yes	No	Keys: No
Indexable <i>access by position/key</i>	Yes	Yes	No	By key

Lists are the most common and versatile data structure. They are ordered, mutable, and allow duplicates. You can put almost anything in a list..

```
fruits = ["apple", "banana", "cherry", "orange"]

# Indexing and slicing
fruits[0]      # "apple"
fruits[-1]     # "orange"
fruits[1:3]    # ["banana", "cherry"]

# Modifying
fruits.append("grape")      # add to end
fruits.insert(1, "kiwi")    # insert at position
fruits.remove("banana")     # remove by value
popped = fruits.pop()       # remove and return last item

# Useful operations
len(fruits)                # number of items
"apple" in fruits           # True – membership test
sorted(fruits)              # returns a new sorted list
```

Dictionaries store **key-value pairs** - just like a real dictionary. Use them when you need to look things up by name (allows fast and precise access).

```
fruit = {  
    "name": "Apple",  
    "sweetness": 7.5,  
    "colors": ["red", "green", "yellow"],  
}  
  
# Access by key  
fruit["name"]           # "Apple"  
fruit.get("origin", "unknown") # "unknown" – safe access with default  
  
# Modify  
fruit["origin"] = "Valais"    # add or update a key  
del fruit["sweetness"]       # remove a key  
  
# Iterate  
for key, value in fruit.items():  
    print(f"{key}: {value}")
```

Tuples are **immutable** lists — once created, they cannot be changed. They are ordered and allow duplicates, but you cannot add, remove, or modify elements.


```
# Creation and indexing
coords = (47.37, 8.54)
coords[0]          # 47.37
coords[-1]         # 8.54 – negative indexing works
coords[0:2]        # (47.37, 8.54) – slicing returns a tuple

# Unpacking (multiple assignment)
lat, lon = coords          # parallel assignment
name, *scores, final = ("Alice", 85, 90, 88, 92) # * collects middle values

# Tuples as dict keys (immutable = hashable)
grid = {(0, 0): "origin", (1, 0): "east", (0, 1): "north"}

# Tuple methods
data = (1, 2, 2, 3, 2)
data.count(2)      # 3 – how many times 2 appears
data.index(3)      # 3 – position of first occurrence
```

Sets store **unique** items in no particular order. They are mutable (you can add/remove items), but duplicates are automatically removed.

```
# Creation – duplicates automatically removed
toppings = {"cheese", "tomato", "cheese"}
print(toppings)      # {"cheese", "tomato"}

# Create from list (useful for deduplication)
responses = set(["yes", "no", "yes", "maybe", "yes"]) # {"yes", "no", "maybe"}

# Add and remove
toppings.add("basil")
toppings.add("basil")      # adding duplicate does nothing
toppings.remove("cheese")  # KeyError if not found
toppings.discard("onion")  # safe – no error if missing

# Membership test (O(1) – very fast!)
"tomato" in toppings      # True
"mushroom" not in toppings # True
```

Use sets when: you need uniqueness, fast membership testing, or set math (overlap, differences).

```
# Set operations
a = {1, 2, 3}
b = {2, 3, 4}
a & b          # {2, 3} – intersection (in both)
a | b          # {1, 2, 3, 4} – union (in either)
a - b          # {1} – difference (in a but not b)
a ^ b          # {1, 4} – symmetric difference (from both sides, but not in both)
```

When in doubt, ask yourself:

Do I need **key-value** pairs? → **dict**
Do I need to prevent changes? → **tuple**
Do I need only unique items? → **set**
Otherwise → **list**

This is a simplification. As you gain experience, you'll develop intuition for subtler trade-offs — like performance with large datasets, or using tuples as dict keys.

MiniTask: Pick the right structure

For each scenario below, choose the **best** data structure and create the variable:

1. `survey_responses` — You're collecting “yes”/“no”/“maybe” responses from 100 students. Some students submit twice by accident. You only want unique responses for counting.
2. `rgb_color` — Store RGB values (e.g., 47, 128, 200) that define a color. These values are always exactly three numbers and should never be modified after creation.

3. `shopping_cart` — Track items someone wants to buy. They might add duplicates, want to see them in order added, and occasionally remove specific items.
4. `country_codes` — Look up country codes by country name (e.g., “Switzerland” → “CH”, “Germany” → “DE”). Fast lookups are essential.

Challenge: Can you justify why each structure is better than the alternatives for its use case?

Operation	List	Tuple	Dict	Set
Indexing [i]	Yes	Yes	By key	No
Slicing [a:b]	Yes	Yes	No	No
Membership in	Yes	Yes	Keys	Yes
Length len()	Yes	Yes	Yes	Yes
Add items	Yes	No	Yes	Yes
Remove items	Yes	No	Yes	Yes



Note:

Tuples are
new ones.

— you cannot add, remove, or change items in place. You can only create

Mutable vs. immutable — demo

Try this out and find the difference between lists and tuples. What gets printed in each case?

```
# Example 1: Lists  
a = [1, 2, 3]  
b = a  
b.append(4)  
print(a)
```

- A) [1, 2, 3]
- B) [1, 2, 3, 4]
- C) Error

```
# Example 2: Tuples  
x = (1, 2, 3)  
y = x  
y += (4,)  
print(x)
```

- A) (1, 2, 3)
- B) (1, 2, 3, 4)
- C) Error

Why the difference?

Lists are mutable

Changes happen **in place**.
Both a and b point to the
same list object in memory.

```
a = [1, 2, 3]
b = a          # b → same
                object
b.append(4)     # modifies THE
                list
print(a)        # [1, 2, 3, 4]
```

Tuples are immutable

`+=` creates a **new** tuple. y
gets reassigned, but x still
points to the original.

```
x = (1, 2, 3)
y = x          # y → same
                object
y += (4,)      # y → NEW
                tuple
print(x)       # (1, 2, 3)
```

- Passing a **list** to a function → the function can modify your original list
- Passing a **tuple** → safe, no accidental modification
- Only **immutable** types can be dictionary keys or set elements

Comprehensions — a shorter syntax

You'll often see this more compact way to build lists in Python code and AI output:

```
# Loop approach
squared = []
for n in range(5):
    squared.append(n ** 2)

# Same thing as a comprehension
squared = [n ** 2 for n in range(5)]
```

The pattern: `[expression for item in iterable]`

Add a filter: `[expression for item in iterable if condition]`

MiniTask: Loop comprehension

Convert this for loop into a list comprehension:

```
# Given list of temperatures in Celsius, convert to Fahrenheit
temps_celsius = [0, 10, 20, 25, 30, 35]
temps_fahrenheit = []
for temp in temps_celsius:
    temps_fahrenheit.append(temp * 9/5 + 32)
```

Write the same logic as a one-line list comprehension.

Bonus: Now filter to only include temperatures above 75°F.

Functions

What is a function?

A **function** is a reusable block of code that performs a specific task.

```
def greet(name: str) -> str:
    """Return a greeting for the given name."""
    return f"Hello, {name}!"

message = greet("Alice")
print(message)  # Hello, Alice!
```

What is a function?

A **function** is a reusable block of code that performs a specific task.

```
def greet(name: str) -> str:
    """Return a greeting for the given name."""
    return f"Hello, {name}!"

message = greet("Alice")
print(message)  # Hello, Alice!
```

A function has four parts:

1. **Name** — what it's called (greet)
2. **Parameters** — what it takes in (name: str)
3. **Body** — what it does (return f"Hello, {name}!")
4. **Return value** — what it gives back (-> str)


```
def celsius_to_fahrenheit(celsius: float) -> float:
    """Convert a temperature from Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32

# Calling the function
temp_f = celsius_to_fahrenheit(25.0)
print(f"25°C = {temp_f}°F") # 25°C = 77.0°F
```

```
def celsius_to_fahrenheit(celsius: float) -> float:
    """Convert a temperature from Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32

# Calling the function
temp_f = celsius_to_fahrenheit(25.0)
print(f"25°C = {temp_f}°F") # 25°C = 77.0°F
```

Key points:

- def starts the definition, colon : ends the signature
- The body is **indented** (just like if and for)
- return sends a value back to the caller — without it, the function returns None

Function arguments: positional, keyword, default

```
def make_order(item: str, quantity: int = 1, size: str = "medium") -> str:
    """Return a formatted order string."""
    return f"Place order for {quantity}x {size} {item}"
```

Positional – order matters

```
make_order("coffee", 2, "large")
```

Keyword – order doesn't matter

```
make_order("tea", size="small", quantity=3)
```

Default – omit arguments that have defaults

```
make_order("water") # uses quantity=1, size="medium"
```

Some good practices for function parameters

- Use **keyword arguments** to avoid passing values to the wrong parameter
- Use **defaults** for parameters that may often have the same value, or to make some parameters optional
- Put parameters **without** defaults before those **with** defaults
- Use **type hints** to clarify what types of values the function expects and returns
- Avoid mutable default arguments (like lists or dicts) — they can lead to unexpected behavior

MiniTask: Tip calculator

Write a function that calculates how much each person owes after a meal:

Required: the bill amount

Optional:

- tip percentage (default: 15)
- number of people splitting (default: 1)
- currency symbol (default: "CHF")

MiniTask: Write a function with defaults

Return a string like: "Each person pays 49.45 CHF"

Test: `split_bill(85.50)`, `split_bill(120, split=4)`,
`split_bill(50, tip=20, currency="EUR")`

A **docstring** is a string on the first line of a function body. It explains what the function does.

```
def my_function():  
    """This is a docstring – it describes the function."""  
    pass  
  
help(my_function)    # prints the docstring  
my_function.__doc__  # accesses it directly
```

- They are the **first thing** another developer (or future you) reads
- `help()` and IDEs display them automatically
- AI tools use them to understand your code
- They are **not** comments — they are part of the function object

Google style

```
def mean(values: list[float]) -  
> float:  
    """Compute the arithmetic  
    mean.  
  
    Args:  
        values: List of numbers  
    to  
        average.  
  
    Returns:  
        The arithmetic mean.  
  
    Raises:  
        ValueError: If the list  
        is empty.  
    """
```

NumPy style

```
def mean(values: list[float]) -  
> float:  
    """Compute the arithmetic  
    mean.  
  
    Parameters  
    -----  
    values : list[float]  
        List of numbers to  
    average.  
  
    Returns  
    -----  
    float  
        The arithmetic mean.  
    """
```


There is no single “correct” format but these two conventions are widely used and recognized. Which one should you use?

- **Google style** — compact, popular in web/general Python projects
- **NumPy style** — common in scientific Python (NumPy, SciPy, pandas)



Note:

Pick one style and use it consistently throughout your project. Both are valid and consistency is what matters.

Variable scope

Scope and the LEGB rule

Scope defines where a variable can be accessed. Python uses the **LEGB** rule:

Scope	Where	Example
L ocal	Inside the current function	x inside <code>def f(): x = 1</code>
E nclosing	Inside an outer (nesting) function	Nested functions
G lobal	Top level of the script/module	x = 1 at the top of a file
B uilt-in	Pre-defined by Python	print, len, range

Python searches from **L** $\rightarrow$ **E** $\rightarrow$ **G** $\rightarrow$ **B**. The first match wins.

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"
        print(f'x is: {x}')

    inner()
    print(f'x is: {x}')

outer()
print(f'x is: {x}')
```

Variables in functions

Key rule: Assignment inside a function creates a **local** variable. It does not change the outer one.



Note:

You rarely need `global` or `nonlocal` keywords. If you find yourself reaching for them, consider passing values as arguments and returning results instead.

File Handling

Python's built-in `open()` lets you read and write files. Always use the `with` keyword — it closes the file automatically.

```
# Writing to a file
with open("shopping.txt", "w") as f:
    f.write("milk\n")
    f.write("bread\n")
    f.write("eggs\n")

# Reading from a file
with open("shopping.txt", "r") as f:
    for line in f:
        print(line.strip()) # .strip() removes the trailing newline
```

Mode	What it does
"r"	Read (default) — file must exist
"w"	Write — creates file or overwrites existing content
"a"	Append — adds to end of file
"x"	Exclusive creation — fails if file already exists

Structured data: JSON and pickle

JSON — human-readable, cross-language

```
import json

data = {"name": "Pancakes",
        "servings": 4}

with open("data.json", "w") as f:
    json.dump(data, f, indent=4)

with open("data.json", "r") as f:
    loaded = json.load(f)
```

Pickle — any Python object, binary

```
import pickle

data = {"coords": (47.37, 8.54)}

with open("data.pkl", "wb") as f:
    pickle.dump(data, f)

with open("data.pkl", "rb") as f:
    loaded = pickle.load(f)
```



Note:

JSON supports dicts, lists, strings, numbers, booleans, and `null`. For Python-specific types (tuples, sets, custom objects), use pickle.

Format	Module / Library	Use case
.json	json	Structured data, APIs, config files
.csv / .xlsx	csv / pandas	Tabular data, spreadsheets
.toml	tomllib	Config files (e.g., pyproject.toml) — Python 3.11+
.pkl	pickle	Python objects (binary, not portable)
.npy / .npz	numpy	Arrays — <code>np.save()</code> , <code>np.load()</code>
.h5 / .hdf5	h5py	Large numerical datasets
.nc	xarray	Multidimensional scientific data (NetCDF)

MiniTask: Read and write data

1. Create a list of dictionaries representing your favourite movies: each with keys "title", "year", and "rating".
2. Write it to a JSON file using `json.dump()` with `indent=4`.
3. Read it back with `json.load()` and print each movie's title.
4. *Bonus:* Save the same data with `pickle` and verify you can reload it.

Putting it all together

The 3 R's applied to functions

Putting it all together

Readable

- Descriptive function name
- Docstring
- One function = one job
- Short (< 20 lines)

Reusable

- General parameters (no hardcoded values)
- Return instead of printing
- Use default arguments

Reproducible

- Type hints on params and return
- Handle edge cases
- Fail early with clear errors

MiniTask: Apply the 3 R's

The following script processes a list of student scores. Refactor it using everything from today.

MiniTask: Refactor this script

Putting it all together

```
s = [78, 92, 55, 88, 43, 91, 67, 72, 85, 39]
p = []
fl = []
for x in s:
    if x >= 60:
        p.append(x)
    else:
        fl.append(x)
t = 0
for x in p:
    t = t + x
print(t / len(p))
print(len(fl))
```


Your refactored version should:

1. Use meaningful variable names
2. Extract at least two functions with type hints and docstrings
3. Use a comprehension where it makes sense
4. Save the results (average passing score, number of failures) to a JSON file

Try using AI as a starting point (Model C) — generate a solution, then review it against the 3 R's. Does the AI follow them?

Recap

After today, you should be able to:

- **Explain** the 3 R's: Readable, Reusable, reproducible — and why they matter
- **Choose** the right data structure: list, tuple, set, or dict
- **Recognize** list comprehensions as a common Python shorthand
- **Define** functions with parameters, defaults, type hints, and docstrings
- **Document** functions using Google or NumPy docstring conventions
- **Describe** variable scope using the LEGB rule
- **Read and write** files using `with open()`, JSON, and pickle
- **Refactor** a working script into clean, well-structured code

Next week (L9): The Python ecosystem — imports, packages, pip, virtual environments